

Lecture 3

Fundamentals of Numerical Analysis

C. Mou

Department of Mathematics and Statistics
Utah State University

Fall 2026

What we'll do:

- error bound
- look at floating point representation in its basic form
- expose errors of a different form: rounding error
- highlight IEEE-754 standard

Big-Theta

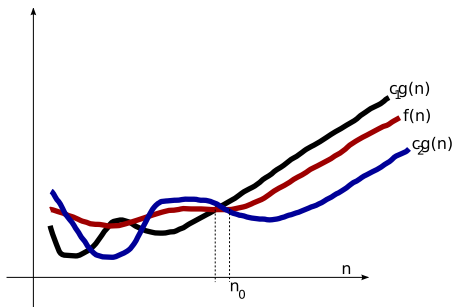
asymptotic tight bound

$\Theta(\cdot)$

Let $g(n)$ be a function of n . Then define

$$\Theta(g(n)) = \{f(n) \mid \exists c_1, c_2, n_0 > 0 : 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n), \forall n \geq n_0\}$$

Equivalently, $\Theta(g(n)) = \mathcal{O}(g(n)) \cap \Omega(g(n))$.



Algebraic Convergence (J. P. Boyd)

Definition

The Algebraic Index of Convergence α is the largest number for which

$$\lim_{n \rightarrow \infty} |a_n| n^\alpha < \infty$$

where a_n are the coefficients in the sequence. Alternatively, α is the algebraic index if

$$a_n \sim \mathcal{O}(1/n^\alpha)$$

Exponential Convergence (J. P. Boyd)

Definition

If the algebraic index α is unbounded (i.e. a_n decrease faster than $1/n^\alpha$ for any finite α), then the sequence converges exponentially (a.k.a. spectrally).

Alternatively, the sequence converges exponentially if for constants q and β

$$a_n \sim \mathcal{O}(e^{-qn^\beta})$$

where β is the exponential index of convergence and

$$\beta = \lim_{n \rightarrow \infty} \frac{\log |\log(|a_n|)|}{\log(n)}$$

Rates of Exponential Convergence (J. P. Boyd)

Definition

A sequence a_n has supergeometric, geometric, or subgeometric if

$$\lim_{n \rightarrow \infty} \log(|a_n|)/n = \begin{cases} \infty, & \text{supergeometric} \\ \text{constant}, & \text{geometric} \\ 0, & \text{subgeometric} \end{cases}$$

or, alternatively,

$$a_n \sim \mathcal{O}(e^{-(n/j) \log(n)}) : \text{supergeometric}$$

$$a_n \sim \mathcal{O}(e^{-qn}) : \text{geometric}$$

$$\beta < 1 : \text{subgeometric}$$

Asymptotic Rate of Geometric Convergence (J. P. Boyd)

definition

If a sequence a_n has geometric convergence ($\beta = 1$) so that

$$a_n \sim \mathcal{O}(e^{-n\mu})$$

then the asymptotic rate of geometric convergence is μ . Alternatively,

$$\mu = \lim_{n \rightarrow \infty} \{-\log |a_n|/n\}$$

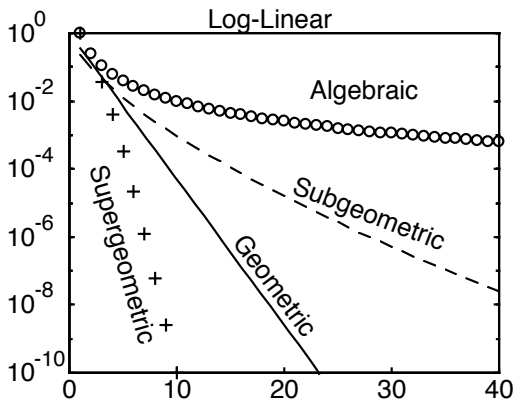


Figure 2.5: $\log |a_n|$ versus n for four rates of convergence. Circles: algebraic convergence, such as $a_n \sim 1/n^2$. Dashed: subgeometric convergence, such as $a_n \sim \exp(-1.5 n^{2/3})$. Solid: geometric convergence, such as $\exp(-\mu n)$ for any positive μ . Pluses: supergeometric, such as $a_n \sim \exp(-n \log(n))$ or faster decay.

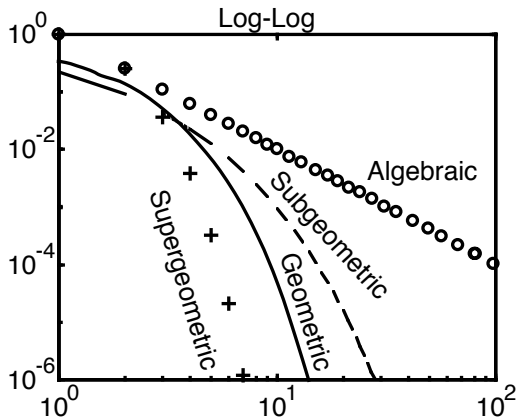


Figure 2.6: Same as previous figure except that the graph is log-log: the degree of the spectral coefficient n is now plotted on a logarithmic scale, too.

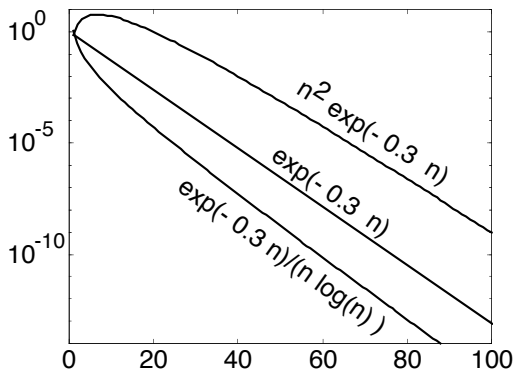


Figure 2.8: Spectral coefficients for three geometrically converging series. Although the three sets of coefficients differ through algebraic coefficients — the top curve is larger by n^2 than the middle curve, which in turn is larger by a factor of $n \log(n)$ than the bottom curve — the *exponential* dependence on n is the same for all. Consequently, all three sets of coefficients asymptote to parallel lines on this log-linear plot.

But Taylor Series Approximations have many terms...

- For example, how do we evaluate

$$f(x) = 5x^3 + 3x^2 + 10x + 8$$

at the point $1/3$?

- This would require 6 multiplications and 3 additions.
- If we regroup as

$$f(x) = 8 + x(10 + x(3 + x(5)))$$

then we have 3 multiplications and 3 additions.

- This is Nested Multiplication or Synthetic Division or Horner's Method

Nested Multiplication

- To evaluate

$$f(x) = a_0 + a_1x + a_2x^2 + \cdots + a_nx^n$$

rewrite as

$$f(x) = a_0 + x(a_1 + x(a_2 + \cdots + x(a_n)) \dots)$$

Listing 1: nested mult

```
1 p = a_n
2 for i = n - 1 to 0 step -1
3   p = a_i + xp
4 end
```

Why this is important:

- Errors come in two forms: truncation error and rounding error
 - we always have them...
 - case study: Intel
 - our jobs as developers: reduce impact

Next: Floating Point Numbers

- We're familiar with base 10 representation of numbers:

$$1234 = 4 \times 10^0 + 3 \times 10^1 + 2 \times 10^2 + 1 \times 10^3$$

and

$$.1234 = 1 \times 10^{-1} + 2 \times 10^{-2} + 3 \times 10^{-3} + 4 \times 10^{-4}$$

- we write 1234.1234 as an integer part and a fractional part:

$$a_3a_2a_1a_0.b_1b_2b_3b_4$$

- For some (even simple) numbers, there may be an *infinite* number of digits to the right:

$$\pi = 3.14159 \dots$$

$$1/9 = 0.11111 \dots$$

$$\sqrt{2} = 1.41421 \dots$$

Other bases

- So far, we have just base 10. What about base β ?
- binary ($\beta = 2$), octal ($\beta = 8$), hexadecimal ($\beta = 16$), etc
- In the β -system we have

$$(a_n \dots a_2 a_1 a_0 . b_1 b_2 b_3 b_4 \dots)_\beta = \sum_{k=0}^n a_k \beta^k + \sum_{k=0}^{\infty} b_k \beta^{-k}$$

Integer Conversion

An algorithm to compute the base 2 representation of a base 10 integer

$$\begin{aligned}(N)_{10} &= (a_j a_{j-1} \dots a_2 a_1 a_0)_2 \\ &= a_j \cdot 2^j + \dots + a_1 \cdot 2^1 + a_0 \cdot 2^0\end{aligned}$$

Compute $(N)_{10}/2 = Q + R$:

$$\frac{N}{2} = \underbrace{a_j \cdot 2^{j-1} + \dots + a_1 \cdot 2^0}_{=Q} + \underbrace{\frac{a_0}{2}}_{=R/2}$$

Example

Example: compute $(11)_{10}$ base 2

$$11/2 = 5R1 \Rightarrow a_0 = 1$$

$$5/2 = 2R1 \Rightarrow a_1 = 1$$

$$2/2 = 1R0 \Rightarrow a_2 = 0$$

$$1/2 = 0R1 \Rightarrow a_3 = 1$$

So $(11)_{10} = (1011)_2$

The other way...

Convert a base-2 number to base-10:

$$\begin{aligned}(11\,000\,101)_2 \\&= 1 \times 2^7 + 1 \times 2^6 + 0 \times 2^5 + 0 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \\&= 1 + 2(0 + 2(1 + 2(0 + 2(0 + 2(0 + 2(1 + 2(1))))))) \\&= 197\end{aligned}$$

Matlab

```
1 >> dec2bin(197)
2 ans = 11000101
3
4 >> bin2dec('11000101')
5 ans = 197
```

Converting fractions

- straight forward way is not easy
- goal: for $x \in [0, 1]$ write

$$x = 0.b_1b_2b_3b_4\cdots = \sum_{k=1}^{\infty} c_k \beta^{-k} = (0.c_1c_2c_3\cdots)_{\beta}$$

- $\beta(x) = (c_1.c_2c_3c_4\cdots)_{\beta}$
- multiplication by β in base- β only shifts the radix

Fraction Algorithm

An algorithm to compute the binary representation of a fraction x :

$$\begin{aligned}x &= 0.b_1b_2b_3b_4\dots \\ &= b_1 \cdot 2^{-1} + \dots\end{aligned}$$

Multiply x by 2. The integer part of $2x$ is b_1

$$2x = b_1 \cdot 2^0 + b_2 \cdot 2^{-1} + b_3 \cdot 2^{-2} + \dots$$

Example

Example: Compute the binary representation of 0.625

$$2 \cdot 0.625 = 1.25 \quad \Rightarrow \quad b_{-1} = 1$$

$$2 \cdot 0.25 = 0.5 \quad \Rightarrow \quad b_{-2} = 0$$

$$2 \cdot 0.5 = 1.0 \quad \Rightarrow \quad b_{-3} = 1$$

So $(0.625)_{10} = (0.101)_2$

A problem with precision

```
1  $r_0 = x$ 
2 for  $k = 1, 2, \dots, m$ 
3   if  $r_{k-1} \geq 2^{-k}$ 
4      $b_k = 1$ 
5      $r_k = r_{k-1} - 2^{-k}$ 
6   else
7      $b_k = 0$ 
8   end
9 end
```

k	2^{-k}	b_k	$r_k = r_{k-1} - b_k 2^{-k}$
0			0.8125
1	0.5	1	0.3125
2	0.25	1	0.0625
3	0.125	0	0.0625
4	0.0625	1	0.0000

A problem with precision

For other numbers, such as $\frac{1}{5} = 0.2$, an infinite length is needed.

0.2 $\rightarrow$.0011 0011 0011 ...

So 0.2 is stored just fine in base-10, but needs infinite number of digits in base-2

!!!

This is *roundoff* error in its basic form...

Business Day

The New York Times

L D1

THURSDAY, NOVEMBER 24, 1994

Copyright © 1994 The New York Times

BUSINESS Digest

THURSDAY, NOVEMBER 24, 1994

DOW 30 Industrials	DOLLAR vs. Japanese Yen	OIL Hymex Spot	BONDS 30-Year Treasury
3,674.53 -3.36	98.47 Yen +0.24 Yen	\$18.15 +0.33	7.94% -0.08

Companies

A flaw in the Pentium, the top chip made by Intel, can cause inaccurate calculations in certain rare cases, Intel said the chip would not have to be recalled. But many scientists and engineers who depend on precise calculations are concerned. [Page D1.]

Gibson Greetings and Bankers Trust settled a lawsuit over derivatives. Gibson agreed to pay Bankers Trust nearly \$6.3 million, or about 36 percent of the \$27.7 million that Bankers claimed it was owed in the trades between the two. [D1.]

Sorry, Charlie: H. J. Heinz and Leo Burnett are parting ways after 27 years. The Burnett agency created familiar advertising characters like Morris the monkey cat and Charlie the tuna. [D1.]

Pfizer will buy SmithKline Beecham's animal health business for \$1.5 billion in cash. The deal would make Pfizer the world's largest maker of animal drugs, topping Merck. [D4.]

Kmart is seeking ways to save as much as \$80 million in two years in addition to store closings and job cuts. [D4.]

A Federal judge frees a New York brokerage account of a British businessman accused of illegal insider trading. [D4.]

The F.T.C. is seeking data on alliances with Caresmark International, in a broadening of an investigation of relations between drug makers and managed health-care drug distributors. [D4.]

Metallgesellschaft of Germany plans to write down the paper value of its shares by half in an effort to rescue itself. [D4.]

I.B.M., AT&T, Apple Computer and Siemens plan to create a universal communication language for computers. [D5.]

Markets
After a roller-coaster session, stocks ended slightly lower. The Standard & Poor's 500 index fell just 0.36 point, easing investors' concerns that stocks were in for a long, hard descent. [D3.]

A stock slide is not likely to change Federal Reserve policy. Indeed, stocks' strength until now surprised Fed officials. [D8.]

Stock markets fell sharply across Europe as investors took their cue from Wall Street and unwaveringly refused to budge. [D6.]

Flaw Undermines Accuracy of Pentium Chips

By JOHN MARKOFF

Special to The New York Times

SAN FRANCISCO, Nov. 23 — An elusive circuitry error in causing a chip used in millions of computers to generate inaccurate results in certain rare cases, heightening anxiety among many scientists and engineers who rely on their machines for precise calculations.

The flaw, an error in division, has been found in the Pentium, the current top microprocessor of the Intel Corporation, the world's largest chip maker. The chip, in several different configurations, is used in many com-

puters sold for home and business use, including those made by I.B.M., Compaq, Dell, Gateway 2000 and others.

The flaw appears in all Pentium chips now on the market, in certain types of division problems involving more than five significant digits, a mathematical term that can include numbers before and after a decimal point.

Intel declined to say how many Pentium chips it made or sold, but Dataquest, a market research company in San Jose, Calif., estimated that in 1994 Intel would sell 5.5 million to 6 million Pentiums, roughly 10 percent of the number of personal

computers sold worldwide.

Intel said yesterday that it did not believe the chip needed to be recalled, assuring that the typical user would have but one chance in more than nine billion of encountering an inaccurate result as a consequence of the error, and thus there was no noticeable consequence to users of business or home computers. Indeed, the company said it was continuing to send computer makers Pentium chips built before the problem was detected.

William Kahan of the University of California at Berkeley, one of the nation's experts on computer mathematics, expressed skepticism about

Intel's contention that the error would only occur in extremely rare instances.

"These kinds of statistics have to cause some wonderment," he said. "They are based on assumptions about the probability of events whose probability we don't know."

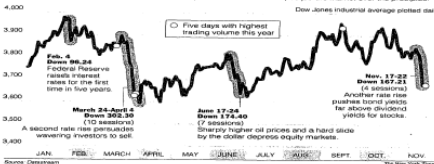
At the Jet Propulsion Laboratory in Pasadena, Calif., one satellite communications researcher who learned of the error this week, said six Pentium machines were used in his group and their use had been suspended for now.

"The Pentium appeared as a con-

tinued on Page D5

Four Sharp Corrections in the Dow

This year the stock market has tended to gain gradually for weeks or bob nervously around a trading range with little direction, waiting for the moment when some bad news or unfavorable trend sets off a headstrong flight. Then, once prices have been beaten down for a few days, the buyers' response and the bleeding stops. Four of these corrections are highlighted, along with the development that traders said pushed the market over the precipice.



Gibson Suit On Trades Is Settled

Bankers Trust Gets 30% of Debt Claimed

By MICHAEL QUINN

Gibson Greetings Inc. and the Bankers Trust Company said yesterday that they had settled Gibson's lawsuit accusing the bank of improperly leading the company to engage in risky financial trades.

Under the out-of-court settlement, Gibson will pay Bankers Trust nearly \$6.3 million, or about 36 percent of the \$27.7 million that Bankers Trust contended it was owed under the trades.

While neither side boasted of victory, Douglas Kidd, a spokesman for Bankers Trust, acknowledged that

tium Chips

In some complex division problems, annoying errors.

corrected.

Some computer users said they believed that Intel had not acted quickly enough after discovering the error.

"Intel has known about this since the summer; why didn't they tell anyone?" said Andrew Schulman, the author of a series of technical books on PC's. "It's a hot issue, and I don't think they've handled this well."

The company said that after it discovered the problem this summer, it ran months of simulations of different applications, with the help of outside experts, to determine whether the problem was serious.

The Pentium error occurs in a portion of the chip known as the floating point unit, which is used for extremely precise computations. In rare cases, the error shows up in the result of a division operation.

Intel said the error occurred because of an omission in the translation of a formula into computer

Close, but Not Close Enough

The owners of computers that use Intel's Pentium microprocessors have found that the chips sometimes do not perform division calculations accurately enough.

The problems arise when the chip has to round a number in a preliminary calculation to get the final result, a task that all processors normally perform. In these cases, however, the Pentium's figures are exact to only 5 digits, not 16, as are those of other computer processors. The Pentium's error, while small, can be 10 billion times as large as those of most chips.

Here is an example of the way the imprecise rounding changes the results of a calculation and the way the deviation from the expected result is calculated.

PROBLEM

$$4,195,835 - [(4,195,835 + 3,145,727) \times 3,145,727]$$

CORRECT CALCULATION

$$= 4,195,835 - [(1.3338204) \times 3,145,727] = 0$$

PENTIUM'S CALCULATION

$$= 4,195,835 - [(1.3337391) \times 3,145,727] = 256$$

DEVIATION

$$256 \div 4,195,835 = 6.1 \times 10^{-5}, \text{ or } 61/100,000$$

Source: Cleve Moler, the Mathworks Inc.

Intel Timeline

from emery.com

June 1994 Intel engineers discover the division error. Managers decide the error will not impact many people. Keep the issue internal.

June 1994 Dr Nicely at Lynchburg College notices computation problems

Oct 19, 1994 After months of testing, Nicely confirms that other errors are not the cause. The problem is in the Intel Processor.

Oct 24, 1994 Nicely contacts Intel. Intel duplicates error.

Oct 30, 1994 After no action from Intel, Nicely sends an email

Intel Timeline

from emery.com

FROM: Dr. Thomas R. Nicely
Professor of Mathematics
Lynchburg College
1501 Lakeside Drive
Lynchburg, Virginia 24501-3199

Phone: 804-522-8374
Fax: 804-522-8499
Internet: nicely@acavax.lynchburg.edu

TO: Whom it may concern

RE: Bug in the Pentium FPU

DATE: 30 October 1994

It appears that there is a bug in the floating point unit (numeric coprocessor) of many, and perhaps all, Pentium processors.

In short, the Pentium FPU is returning erroneous values for certain division operations. For example,

$0001/824633702441.0$

is calculated incorrectly (all digits beyond the eighth significant digit are in error). This can be verified in compiled code, an ordinary spreadsheet such as Quattro Pro or Excel, or even the Windows calculator (use the scientific mode), by computing

$00(824633702441.0)*(1/824633702441.0)$,

which should equal 1 exactly (within some extremely small rounding error; in general, coprocessor results should contain 19 significant decimal digits). However, the Pentiums tested return

0000.999999996274709702

.
.
.

Intel Timeline

from emery.com

- Nov 1, 1994 Software company Phar Lap Software receives Nicely's email. Sends to colleagues at Microsoft, Borland, Watcom, etc. decide the error will not impact many people. Keep the issue internal.
- Nov 2, 1994 Email with description goes global.
- Nov 15, 1994 USC reverse-engineers the chip to expose the problem. Intel still denies a problem. Stock falls.
- Nov 22, 1994 CNN *Moneyline* interviews Intel. Says the problem is minor.
- Nov 23, 1994 The MathWorks develops a fix.
- Nov 24, 1994 New York Times story. Intel still sending out flawed chips. Will replace chips only if it caused a problem in an important application.
- Dec 12, 1994 IBM halts shipment of Pentium based PCs
- Dec 16, 1994 Intel stock falls again.
- Dec 19, 1994 More reports in the NYT: lawsuits, etc.
- Dec 20, 1994 Intel admits. Sets aside \$420 million to fix.

Numerical "bugs"

Obvious

Software has bugs

Not-SO-Obvious

Numerical software has two unique bugs:

- 1 roundoff error
- 2 truncation error

Numerical Errors

Roundoff

Roundoff occurs when digits in a decimal point ($0.3333\dots$) are lost (0.3333) due to a limit on the memory available for storing one numerical value.

Truncation

Truncation error occurs when discrete values are used to approximate a mathematical expression.

Uncertainty: well- or ill-conditioned?

Errors in input data can cause *uncertain* results

- input data can be experimental or rounded. leads to a certain variation in the results
- *well-conditioned*: numerical results are insensitive to small variations in the input
- *ill-conditioned*: small variations lead to drastically different numerical calculations (a.k.a. poorly conditioned)

Our Job

As numerical analysts, we need to

- 1 solve a problem so that the calculation is not susceptible to large roundoff error
- 2 solve a problem so that the approximation has a *tolerable* truncation error

How?

- incorporate roundoff-truncation knowledge into
 - the mathematical model
 - the method
 - the algorithm
 - the software design
- awareness → correct interpretation of results

Floating Points

Normalized Floating-Point Representation

Real numbers are stored as

$$x = \pm(0.d_1d_2d_3 \dots d_m)_\beta \times \beta^e$$

- $d_1d_2d_3 \dots d_m$ is the mantissa, e is the exponent
- e is negative, positive or zero
- the general normalized form requires $d_1 \neq 0$

Floating Point

Example

In base 10

- 1000.12345 can be written as

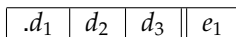
$$(0.100012345)_{10} \times 10^4$$

- 0.000812345 can be written as

$$(0.812345)_{10} \times 10^{-3}$$

Floating Point

Suppose we have only 3 bits for a mantissa and a 1 bit exponent stored like



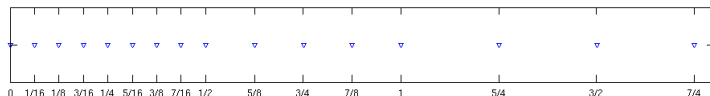
All possible combinations give:

$$000_2 = 0$$

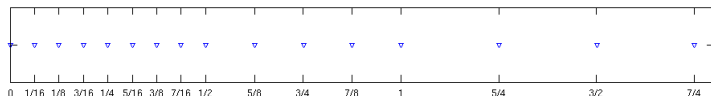
$$\dots \times 2^{-1,0,1}$$

$$111_2 = 7$$

So we get $0, \frac{1}{16}, \frac{2}{16}, \dots, \frac{7}{16}, 0, \frac{1}{4}, \frac{2}{4}, \dots, \frac{7}{4}$, and $0, \frac{1}{8}, \frac{2}{8}, \dots, \frac{7}{8}$. On the real line:



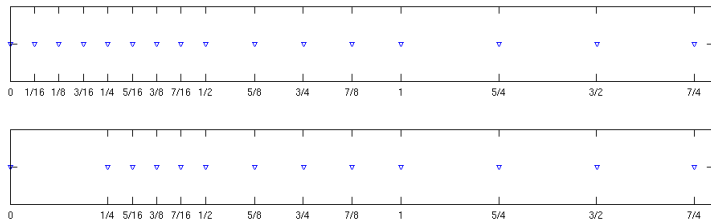
Overflow, Underflow



- computations too close to zero may result in *underflow*
- computations too large may result in *overflow*
- overflow error is considered more severe
- underflow can just fall back to 0

Normalizing

If we use the normalized form in our 4-bit case, we lose $0.001_2 \times 2^{-1,0,1}$ along with other. So we cannot represent $\frac{1}{16}$, $\frac{1}{8}$, and $\frac{3}{16}$.



More on numerical representation...

- read

5th ed. end of section §2.1 in NMC

6th ed. appendix B.1 in NMC

Next:

- Floating Points

- IEEE 754
- mantissa, exponent, machine epsilon, hidden bits
- roundoff, rounding

- loss of significance

- precision